

C++ | CPl

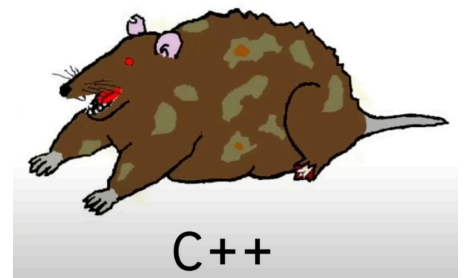
Summary

CONTENTS

1. Why C++?	2
2. Intro	2
2.1. Compilation	2
2.2. Declarations and definitions	3
2.2.1. #include guards	3
2.2.2. Functions	3
2.2.3. Values and references	3
2.3. Libraries	4
3. Values and Streams	4
3.1. Variables	4
3.2. Literal Values	4
3.3. Expressions	5
3.3.1. Type Conversion	5
3.3.2. Assignment Operation	5
3.3.3. Logic Operations	5
3.3.4. Floating Point Numbers (IEEE754)	5
3.3.5. Strings	6
3.3.6. Basic Streams	6
Bibliography	8

1. WHY C++?

- Multi-paradigm language, now you have a thousand and one way to shoot yourself in your foot and obliterate your PC
- Being called stupid when you think that the move function actually moves things or if you confuse a const pointer with a pointer of const
- ISO standard: C++23. The higher the number the hotter the mess of poorly designed choices that have accumulated over decades into a chaotic pile of obsolescence
- Good excuse for alcoholism
- Zero-cost abstractions (if you like learning the whole stdlib by heart and don't mind the cost of debugging template errors for hours on end)
- Headaches (if you're a masochist)
-
- It's a never ending journey. Even if you dedicate your whole life to learning C++, you'll never reach the day where you'll know all of the stdlib.
- "Undefined behavior" is defined (in the standard)
- Ever wanted to have 100 different ways to do the same thing? Now you can
- Has a pretty mascot [\[1\]](#)



2. INTRO

- Implicitly returns 0
- Return types are written in front of the function name or as trailing return-types in declarations (because who needs consistency, amirite)

Nice and clean: `int main() { }` Insanity: `auto main() → int { }`

- Header files can be either `*.hpp` or `*.h`
- Regarding include order: It is good practice to include own-headers ("`"`") before system-headers ("`<`"). The first is usually used for the header files belonging to the project itself, while the latter is used for includes of the standard library and external libraries. However, ***this difference is only conventional, as any toolchain could just specify their own lookup rules for the two distinct kinds of includes.***

Do's	Don'ts
	Using global variables
	Redundant returns that would be implicit
	<code>using namespace</code>
	Not initializing a variable upon definition
	Not using <code>const</code> enough
	TODO: make this column real big for the lulz

2.1. COMPILATION

3 Phases of compilation:

Preprocessor: Textual replacement of preprocessor directives (`#include`)

Compiler: Translation of C++ code into machine code (source file to object file)

Linker: Combination of object files and libraries into libraries and executables

2.2. DECLARATIONS AND DEFINITIONS

All things with a name that you use in a C++ program must be declared before you can do so

- E.g. a function that you call (major difference from C)
- A type that you use for a variable (except some built-ins)
- A variable that you use

There can only be one definition of the same function (**One Definition Rule**/ODR), in contrast to declaration, which can be done several times.

2.2.1. `#include` guards

`#include` guards ensure that a header file is only included once. Necessary for functions and class type definitions, good practice either way.

```
#ifndef GUARD_HPP_
#define GUARD_HPP_
// ...
#endif /* GUARD_HPP_ */
```

2.2.2. Functions

Declarations

```
auto <function-name>(<parameters>) → <return-type>;
<return-type> <function-name>(<parameters>);
```

Definitions

```
auto <function-name>(<parameters>) → <return-type> { /*body*/ }
<return-type> <function-name>(<parameters>) { /*body*/ }
```

If a function has a non-void return type it must return a value on every path (or throw an exception).

2.2.3. Values and references

```
struct Point {
    int x;
    int y;
};
```

Copying vs. Sharing:

```
Point point{1, 20};
```

```
Point copiedPoint{point};
```

```
Point &sharedPoint{point};
```

```
auto copydPointParam(Point point) → Point { }
```

```
auto referencePointParam(Point & point) → Point { }
```

2.3. LIBRARIES

- [catch2](#)
- [boost](#)

TODO:

cmake

3. VALUES AND STREAMS

3.1. VARIABLES

```
<type> <variable-name>{<initial-value>;
```

Using = or {} for initialization with a value supplied we can have the compiler determine its type.

```
auto const i = 5;
```

Initialization might be omitted but that is bad practice and potentially dangerous.

```
double x;
```

Adding the const keyword in front of the name makes the variable a single-assignment variable, aka a constant which is immutable and must be initialized. Some constants are required to be fixed at compile time (**constexpr**). The keyword **const** also appears in other contexts.

```
int const theAnswer{42};
double constexpr pi{3.14};
```

You should use **const** whenever possible for non-member variables.

3.2. LITERAL VALUES

<i>Literal Example</i>	<i>Type</i>	<i>Value</i>
'a'	char	Letter a, value: 97
'\n'	char	<NL> character, value: 10
'\x0a'	char	<NL> character, value: 10
1	int	1
42L	long	42
5LL	long long	5
int{} // (not really a literal)	int	0 (default value)
1u	unsigned int	1
42uL	unsigned long	42
5uLL	unsigned long long	5
020	int	16 (octal 20)
0x1f	int	31 (hex 1F)
0XFULL	unsigned long long	15 (hex F)
0.f	float	0

<code>.33</code>	double	0.33
<code>1e9</code>	double	1000000000 (10 ⁹)
<code>42.E-12L</code>	long double	0.00000000042 (42*10 ⁻¹²)
<code>.3l</code>	long double	0.3
<code>"hello"</code>	char const [6]	Array of 6 chars: h e l l o <NUL>
<code>"\012\n\\"</code>	char const [4]	Array of 4 chars: <NL> <NL> <NUL>

3.3. EXPRESSIONS

	<i>Arithmetic</i>	<i>Bit-operators</i>	<i>Logic</i>
<i>Unary</i>	<code>+ - ++ --</code>	<code>~</code>	<code>!</code>
<i>Binary</i>	<code>+ - * / %</code>	<code>& ^ << >></code>	<code>&& < > ≤ ≥ = ≠</code>
<i>Tertiary</i>			<code>? :</code>

- Fraction results of integer operations are always rounded down
- [Operator precedence](#)

3.3.1. Type Conversion

- Integer to boolean conversion `0` → `false` / every other value → `true`
- Automatic type conversion if values of different types are combined in an expression, **unless in braced initialization**
- Dividing integers by zero is **undefined behavior**

3.3.2. Assignment Operation

```
// vvvvv lvalue
    myVar = 6 * 7;
//          ^^^^^ rvalue
```

3.3.3. Logic Operations

Logical operators and conditional statements are generous to accept numeric values as statement of truth

TODO:

```
if (a < b < c);

// three-way-comparison
≤ >
```

3.3.4. Floating Point Numbers (IEEE754)

- Use `double` – usually most efficient on current hardware and default for floating point literals.
- Use `float` only if memory consumption is utmost priority and precision and range can be traded (on 64bit often not beneficial).
- Remember there are legal double values that are not numbers: `NaN`, `+Inf`, `-Inf`.
- Comparing floating points for equality (`==`) is usually wrong
 - In Catch2 you have the option to specify a matcher that checks in a relative range to the expected value


```
REQUIRE_THAT(actual, Catch::Matchers::WithinRel(expected, delta));
```

3.3.5. Strings

`std::string` is C++'s type for representing sequences of `char` (which is often only 8 bit). It is mutable and iterable with iterators.

```
#include <string>
```

```
std::string statement{"Rust ftw"};
```

TODO:

Unicode support?

String literals like `"ab"` are not of type `std::string`, they're a null-terminated array of const characters (`char const[3]`).

But `"ab"`'s is an `std::string` but requires `using namespace std::literals`.

3.3.6. Basic Streams

Streams aren't values, because they cannot be copied. So functions taking a stream object must take it as a reference.

Pre-defined globals: `std::cin` `std::cout`, should only be used in `main()`. "shift" operators read into variables or write values and can be chained:

```
std::cin >> x;
std::cout << "the value is " << x << '\n';
```

TODO:

Streams have a state that denotes if I/O was successful or not.

- Only `.good()` streams actually do I/O
- You need to `.clear()` the state in case of an error

If a previous read already failed, subsequent reads fail as well

Reading a `std::string` can not go wrong, unless the stream is already `!good()`

- The content of the `std::string` is replaced
- Maybe the `std::string` is empty after reading

Reading an `int`:

- No error recovery
- One wrong input puts the stream into status fail
- Characters remain in input

3.3.6.1. Boolean Conversion

```
int age;
if (std::cin >> age) {
    return age;
}
```

Result of `std::cin >> age` is the `istream` object itself. The stream object converts to `bool` (in if and loop conditions):

- `true` if the last reading operation has been successful
- `false` if the last reading operation failed somehow (formatting, stream end or another problem)

3.3.6.2. States

<i>State Bit Set</i>	<i>Query</i>	<i>Entered</i>
<none>	is.good()	initial is.clear()
failbit	is.fail()	formatted input failed
eofbit	is.eof()	trying to read at end of input
badbit	is.bad()	unrecoverable I/O error

TODO:

- Formatted input on stream is must check for is.fail() and is.bad()
- If failed, is.clear() the stream and consume invalid input characters before continue

TODO:

slides 39, 40+

BIBLIOGRAPHY

- [1] "C++ Mascot." Accessed: Sept. 12, 2026. [Online]. Available: https://media.licdn.com/dms/image/v2/D4D22AQFwZZ5wmpZmnw/feedshare-shrink_800/feedshare-shrink_800/0/1724256362408?e=2147483647&v=beta&t=OXxfC_GlaJFqlUgO03e-9B59Zpzu4-LYG90IM8k4UFE